# Software Testing

Oscar Chaparro

# Worked at 8 hours, failed at 100

1991 Gulf War: a US **Patriot** battery misses an Iraqi **Scud**. **28 soldiers die.**

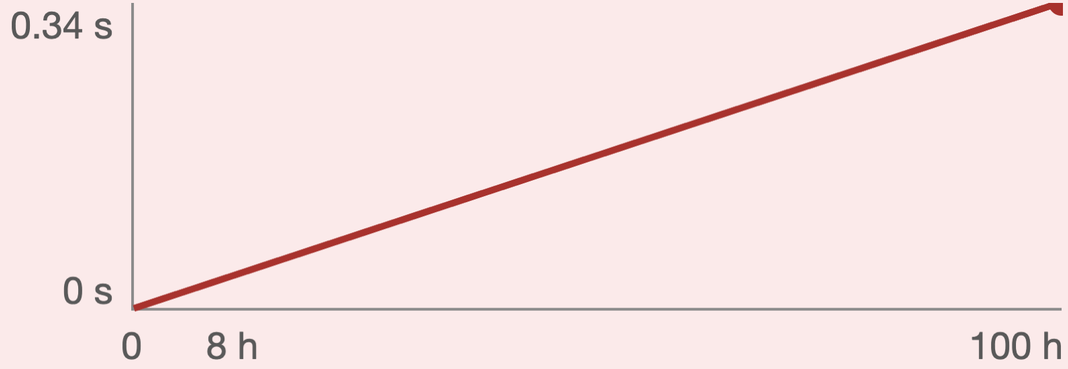
**Timeline, 1991**




Fix released Feb 16 · **arrived Feb 26, the day after**

**Clock error vs. hours running**

Clock counted **tenths of a second**; 0.1 **truncated** to 24 bits → each tick slightly short




After **100+ hours**: clock **0.34 s** off → tracking window **687 m** off

# Why software is hard to test

**Bridge**




**8 h ✓ ⇒ 100 h ✓**

a static load doesn't drift: if it holds for 8 hours, it holds for 100

**Software**



**8 h ✓ ⇏ 100 h ?**

state builds up over time: 8 hours says nothing about 100

**Small changes in inputs, time, or environment → very different behavior**

**"Program testing can be used to show the presence of bugs, but never to show their absence!"** — E. W. Dijkstra, "The Humble Programmer," Communications of the ACM 15(10), 1972

# What else makes testing hard

| Difficulty | Where we cover it |
| --- | --- |
| **Too many inputs:** you can’t try them all | **Designing tests** |
| **Knowing the right answer:** the oracle problem | **Test oracles** |
| **Hidden dependencies:** databases, services, clocks are slow or hard to control | **Test doubles** |
| **State and environment:** time, order, machine change the result | **Suite health** |
| **Change:** every edit can break something far away | **Regression testing** |

# Agenda

1. Foundations
2. Designing tests from the specification
3. Coverage
4. Test oracles in practice
5. Test doubles and integration
6. Keeping a test suite healthy
7. Regression testing and prioritization
8. Mutation testing
9. Fuzzing and property-based testing
10. Testing AI components
11. Agents and testing

# Foundations

What a test is, and what it checks

# Fault, error, failure

```
function double(x: number) {
  return x * x;
}
```

**Fault (bug)**

the wrong code

`*` instead of `+`

→

**Error**

the wrong internal state it creates

`double(3)` computes `9`

→

**Failure**

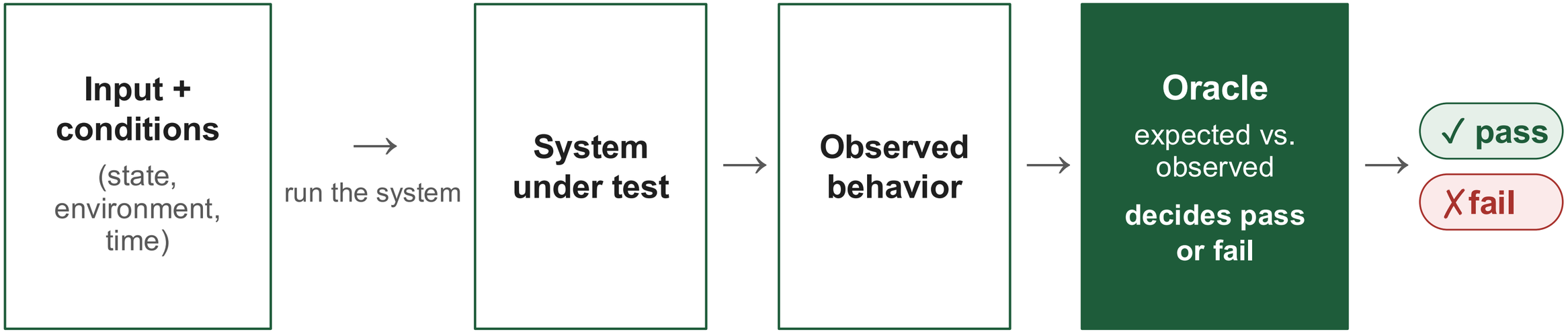
the wrong behavior a user can see

`screen shows 9, expected 6`

**A fault can sit unnoticed until some input turns it into a failure**

`double(2) = 4` — correct by luck.

# What a test is




**No oracle, no test — just a demo**

# Test levels

more realistic, slower,

| Level | Checks | Built from | Typical tools |
|---|---|---|---|
| **Unit** | one function or class | code + module spec | Vitest, Jest, go test |
| **Integration** | parts working together | interface specs | HTTP tests, real database |
| **System / E2E** | the whole app, as a user | requirements | Playwright, Cypress |
| **Acceptance** | does it meet the need? | acceptance criteria | users, product owners |

Levels describe **scope**. At any level you can also test **non-functional** qualities: performance · load · endurance (long runs) · security · accessibility

# What each level checks

## Unit

- return values
- thrown errors
- state changes
- log output

## Integration

- messages between parts
- database contents
- error codes
- contracts and schemas

## E2E (end-to-end)

- screens and transitions
- data shown
- final outcomes
- backend state

**The higher the level, the more there is to observe, and the more ways a check can be too weak or too brittle**

# What makes testing good

## Effective

finds real faults

**How to tell:** break the code on purpose; does a test fail?

## Repeatable

same input, same result — every time

**How to tell:** rerun 5 times; does any test flip?

## Automated

cheap to run often

**How to tell:** does CI run it on every push?

## Systematic

inputs chosen on purpose, not by luck

**How to tell:** can you name what each test covers?

**Which of these does your project's test suite fail today?**

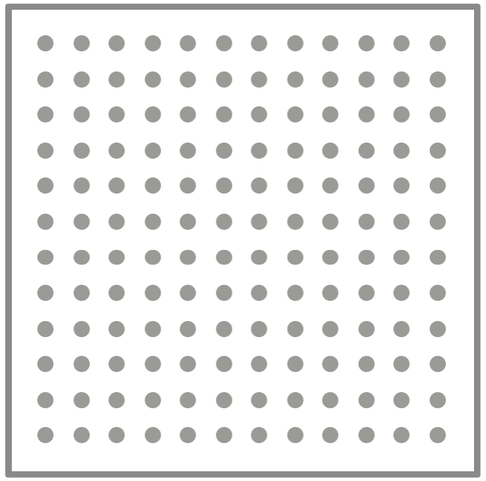
# Designing tests from the specification

Black-box test design

# You can’t test every input

3 inputs × 32 bits each → $(2^{32})^3 = 2^{96} \approx \mathbf{8 \times 10^{28}}$ combinations

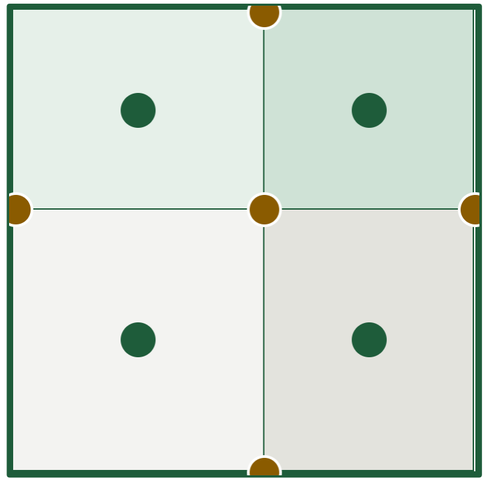
At a billion tests per second: about **2.5 trillion years**



all inputs

→

partition



groups

- **Group inputs that should behave the same**
- **Test one from each group — and the edges between groups**

# Equivalence classes




**nextDate(month, day, year)**, year 1812–2050 → the date of the next day

## Month

- 30-day
- 31-day
- February

## Day

- 1–28
- 29
- 30
- 31

## Year

- common
- leap (÷4)
- 1900 (÷100)
- 2000 (÷400)

invalid: < 1812 · > 2050

# Weak vs. strong combinations

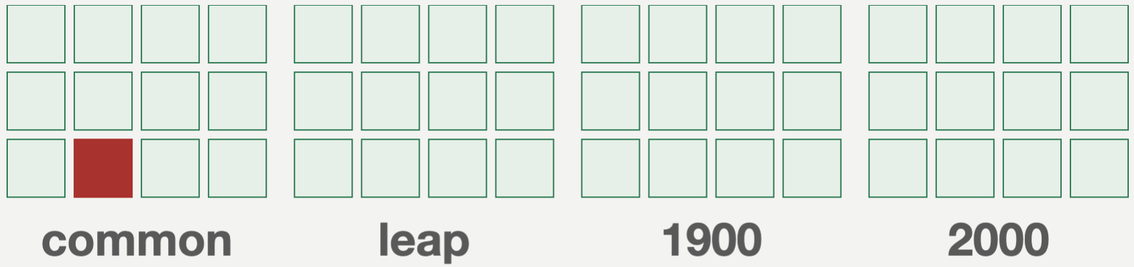
## Weak

every class appears at least once → **4 tests**

| Test | Input | Expected |
|---|---|---|
| W1 | 6/14/1900 | 6/15/1900 |
| W2 | 7/29/1912 | 7/30/1912 |
| W3 | 2/30/1913 | invalid |
| W4 | 6/31/2000 | invalid |

## Strong

every combination of classes → **48 tests**




rows: month · columns: day

**✗ Feb · 29 · common year**

**Weak** is cheap; **strong** finds faults that need two inputs together (Feb 29 in a common year)

# Boundary values

Faults cluster at the **edges** of classes



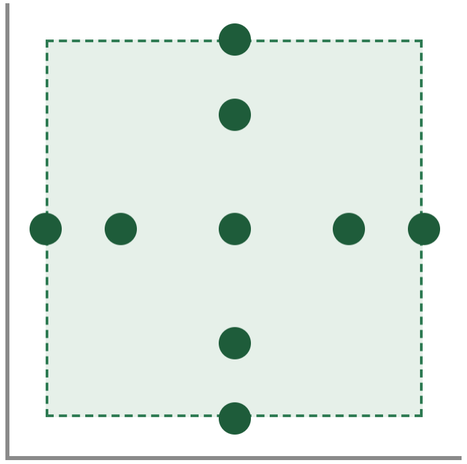

**Normal:** min · min+ · nominal · max− · max

**Robust:** also min− and max+ (invalid)

# Two inputs: normal vs. worst case

## Normal

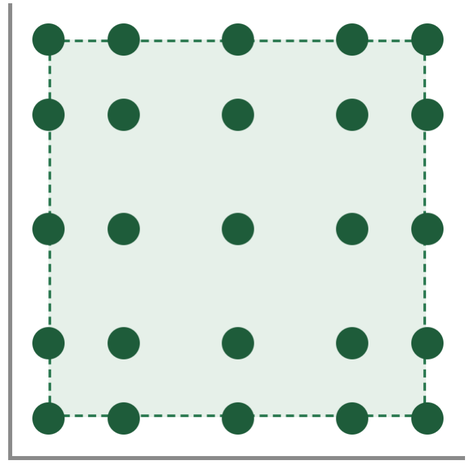
vary one input at a time



**9 tests** (4n + 1)

## Worst case

every combination



**25 tests** ($5^n$)

With 5 inputs: normal = **21** tests · worst case = **3,125**. Use worst case when failures are costly.

# Boundaries for nextDate

| | min | min+ | nom | max− | max |
|---|---|---|---|---|---|
| **month** | 1 | 2 | 6 | 11 | 12 |
| **day** | 1 | 2 | 15 | 30 | 31 |
| **year** | 1812 | 1813 | 1931 | 2049 | 2050 |

**Normal** (13 tests)

`6/31/1931` → **invalid**

`12/15/1931` → **12/16/1931**

**Worst case only** (125 tests)

`12/31/1812` → **1/1/1813**

`12/31/2050` → **out of range**

**Boundaries of the range miss 1900 and 2000: combine with equivalence classes**

# December 31, 2008




Dec 31, 2008: every **Zune 30 GB** (Microsoft's music player) froze on startup, all at once

The clock code counted days in the year. On **day 366** of a leap year, a loop never ended

Microsoft's advice: wait until Jan 1 and restart

**The boundary nobody tested:** the last day of a leap year

Source: Microsoft via NBC News, 2009

# Outline: boundaries in practice

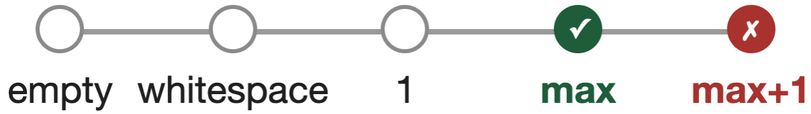
Archive a document with an optional **reason**, up to a maximum length

server/ routes/ api/ documents/ documents.test.ts

```
it("should allow a reason at the maximum length",
  async () => {
    const reason = "a".repeat(
    DocumentValidation.maxArchivedReasonLength);
    …
  });
it("should reject a reason over the maximum length",
  async () => {
    const reason = "a".repeat(
    DocumentValidation.maxArchivedReasonLength + 1);
    …
  });
```

**TESTS IN THE REPO**

- no reason
- empty
- only whitespace
- trims spaces
- **exactly max**
- **max + 1** rejected
- child documents
- restore clears it




The limit comes from a **constant** , not a typed number

# Medusa: dates have edges too

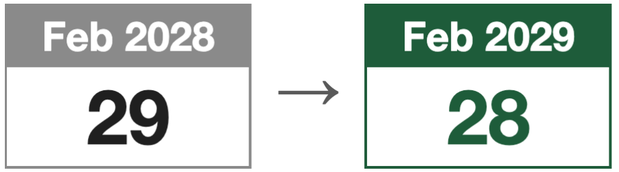
Subscriptions renew weekly, monthly, or yearly

**TESTS IN THE REPO**

```
describe("computeNextBillingDate", () => {
  it("adds 7 days")
  it("adds one month")
  it("clamps to the end of a shorter month")
  it("rolls over to next year")
  it("clamps a yearly interval from a leap day")
  it("doesn't change the input")
});
```




monthly: **no Feb 31** → last day of Feb



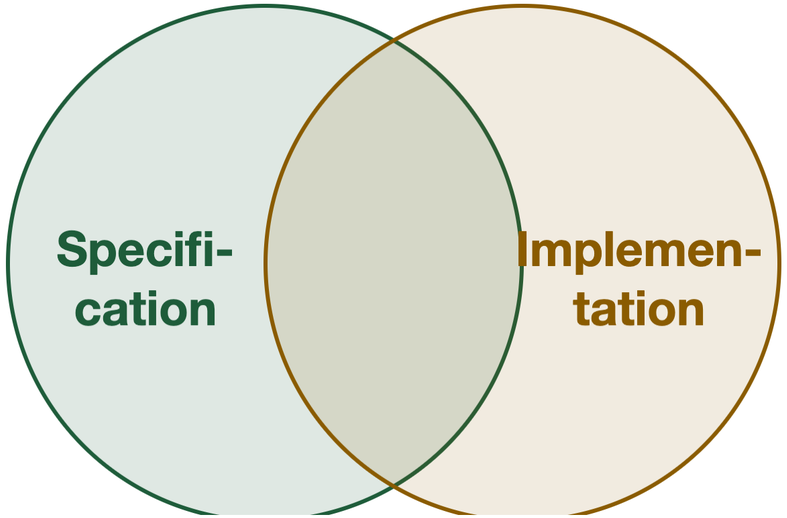

yearly: **no Feb 29, 2029** → Feb 28

**Other boundaries to test:** a monthly plan starting **Feb 29** · a plan from **Jan 31** : after Feb 28, does it renew on Mar 28 or Mar 31?

# What each approach can’t see

**missing functionality**

→ only **black-box** finds it




**unexpected functionality**

→ only **white-box** finds it

**Specification** = what the system should do (requirements, acceptance criteria, API docs)

**Black-box** (from the specification): can’t see code that does something **unexpected**

**White-box** (from the code): can’t see something the specification asked for but **is missing**

## Use both

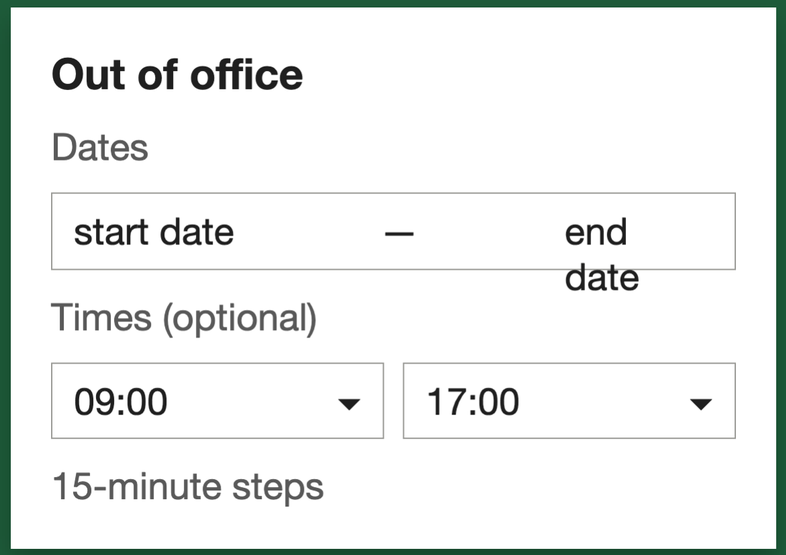
# Exercise: an out-of-office window

**Cal.diy:** a host marks time as “out of office” so no one can book it

**Inputs:** start date · end date · start time · end time (times are optional: no times = whole days)

**Rules:** the end must come after the start · times are “HH:mm” · an end time of 23:59 means end of day

**Your task:** list equivalence classes and boundary values. Which **combinations** matter?



# An out-of-office window — answers

| Input | Classes | Boundaries |
|---|---|---|
| **Times** | absent (whole days) · present | 00:00 · 23:59 ✓ · 24:00 ✗ · “9:00” ✗ |
| **Same day** | end after start ✓ · end before start ✗ | **end = start** ✗ · start + 1 minute ✓ |
| **Different days** | end date after start date ✓ · before ✗ | window across midnight ✓ (23:00 → 02:00) |
| **End time** | ordinary · 23:59 (end of day) | 23:58 vs. 23:59 |

**same day × time order is where the faults are**

**Cal.diy’s tests cover:** end before start · end = start · 23:59 · past midnight. **Not in the handler tests:** time format, which the schema checks instead.

**Also worth testing:** end date = start date · only one time given · DST days · overlapping windows

# Can tools find the classes for you?

Tools can’t read intent. **People choose the classes from the spec**; tools help with the rest.

**You list classes and boundary values → the tool picks combinations**

PICT | ACTS

you map each row to real inputs and the expected result

**The tool reads the code → finds its boundaries**

EvoSuite | Pynguin | KLEE

every `if` becomes a boundary

**You give a rule → the tool searches for inputs that break it**

fast-check | fuzzers

tries thousands of inputs, biased toward edges (0, empty, max)

**Code-based tools can’t find missing classes: they’d never propose day 366 unless the code handles it**

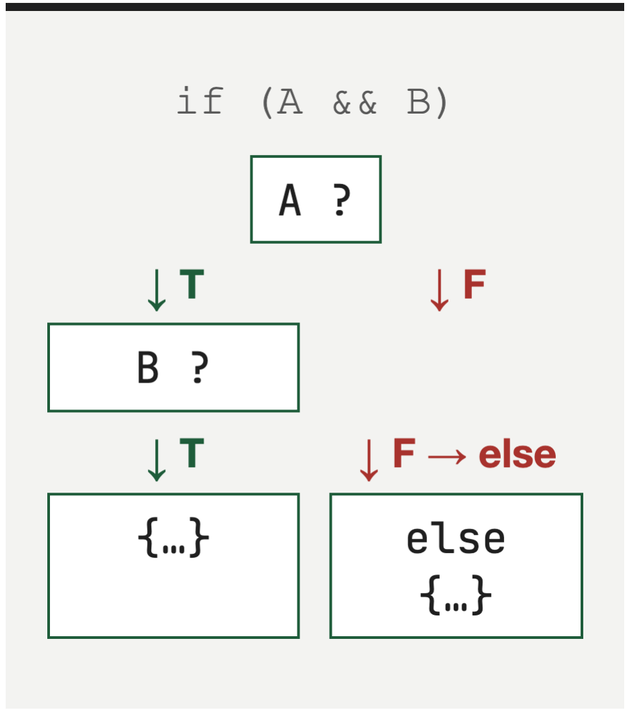
# Coverage

What ran — and what that doesn’t tell you

# Coverage criteria

stronger, costlier

| Criterion | Every … executed at least once |
|---|---|
| **Statement** | line |
| **Branch** | true and false side of each decision |
| **Condition** | true and false value of each part (A, B in A && B) |
| **MC/DC** | each part shown to change the outcome on its own |
| **Path** | route through the code (usually infinite with loops) |



# Apple’s goto fail (2014)

sslKeyExchange.c, Apple Secure Transport

```
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail;
…
err = sslRawVerify(…);
```

always runs

never runs

Checks that the server really owns its certificate. Used by Safari, Mail and apps on iOS and macOS.

- One duplicated `goto fail;` skipped the final signature check
- → fake servers accepted on iOS and macOS
- A **bad-signature** test would fail; coverage shows the check **never ran**

Source: M. Bland, “Finding More Than One Worm in the Apple,” ACM Queue, 2014

# Use coverage to find gaps, not as a goal

1. Write tests from the **specification**
2. Measure **coverage**
3. For each uncovered part: **which requirement does it serve?** → write that test

↺ repeat

```
test("applies the discount", () => {
  const total = applyDiscount(cart, 0.1);
  expect(total).toBeLessThan(100);
});
```

**covered ✓, checked weakly ✗**

a 20% discount or a rounding bug **still passes**

**Measure with:** Vitest `--coverage` (V8 or Istanbul) · patch coverage on PRs (Codecov)

**Coverage shows what ran, not how well it was checked**

# From uncovered code to a requirement

## Safety-critical (DO-178C, avionics)

DO-178C: the standard for certifying aircraft software

Each line or branch the tests never ran must be explained as:

- **Missing test:** add or change a test
- **Missing requirement:** fix it, then test it
- **Dead code:** no requirement: remove it
- **Deactivated code:** show it can't run by accident

## Typical web team

1. Issue lists the criteria
2. PR links the issue
3. Patch coverage shows untested lines
4. Review: which criterion?

**Agent help:** give it the uncovered lines *and* the criteria.

**Tools find what didn't run; people decide why and what to do: a missing test? a missing requirement? code to delete?**

Sources: Ansys; LDRA; Rapita; Codecov docs; Ivanković et al., FSE 2019

# Test oracles in practice

Deciding what “correct” means

# An oracle has two parts

| Oracle | |
|---|---|
| **Information**<br>what the correct behavior is | **Procedure**<br>what to observe, and how to compare |

```
expect(total).toBe(42)
```

**where did 42 come from?**

An oracle is more than an `expect`: snapshots, schemas, reference implementations, invariants, a person's judgment

**A weak oracle is easy to write; a good one is hard: what to expect, what to observe, how to compare**

# Our study: how practitioners engineer test oracles

## Practices, Challenges, and Needs in Functional Test Oracle Engineering: Insights from Practitioners

Anonymous Author(s)

***Abstract*—Functional test oracles are essential for determining whether software behaves as expected. However, engineering functional oracles is challenging because expected behavior is often hard to identify, verify, and maintain as systems evolve. Although prior work has studied test oracles and general software testing practices, little is known about how practitioners engineer functional test oracles in GUI-based systems in practice. We conducted a mixed-method study with 51 software practitioners, including follow-up interviews with 10 participants. We found that practitioners determine expected behavior from many sources, including requirements, project knowledge, UI artifacts, architecture documents, existing behavior, and stakeholder clarification. They verify a wide range of system behavior, from return values and exceptions to UI workflows, backend state, logs, and service interactions. Tools and generative AI support some testing and oracle-related activities, but practitioners still rely heavily on human judgment to define expectations, inspect outputs, verify AI-generated results, and interpret failures. In addition, practitioners struggle with unclear or undocumented expected behavior, frequent changes, high maintenance effort, complex behavior across components or workflows, and conflicting expectations across sources. These findings suggest that oracle engineering should be treated as a first-class engineering activity and that future support should help practitioners capture, assess, maintain, and trace expected behavior knowledge.**

changes, API interactions, logs, and external effects [4, 5]. Thus, practitioners need to reason about both user-facing behavior and underlying system behavior when deciding whether a test passed or failed [6]. Prior work has proposed and evaluated many techniques for GUI testing and test oracles [1, 4, 7–12], and prior empirical studies have examined testing practice more broadly [13–15]. However, we still lack an empirical account of how practitioners engineer functional test oracles in GUI-based systems: how they determine expected behavior, decide what behavior to check, judge correctness, use tools, maintain oracles, and address oracle-related challenges.

In this paper, we study *oracle engineering* as the set of activities involved in determining expected behavior, selecting the system behavior to observe, judging correctness, and maintaining those judgments as systems evolve. Understanding these activities is important for identifying gaps in current support and for guiding future tools, methods, processes, research, and training/education.

To address this gap, we conducted a mixed-method study with 51 software practitioners with diverse roles, experience levels, domains, and project contexts. All participants had

# E2E: Where does the expected value come from?

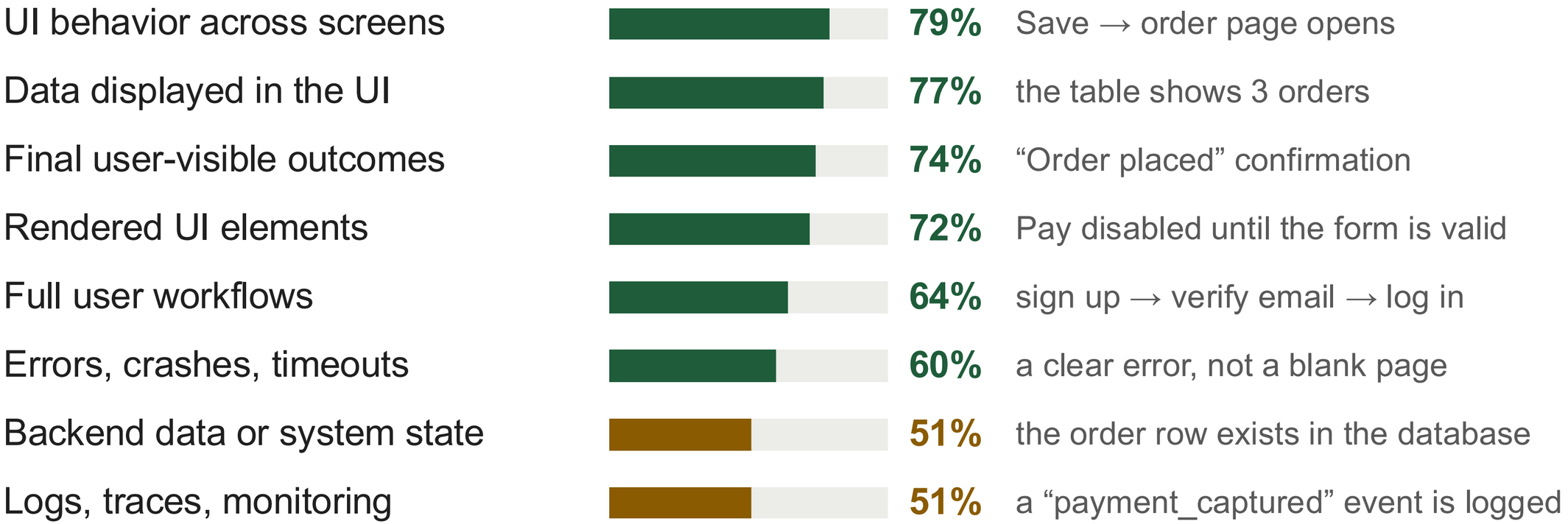



E2E tests · n = 47 · source: our study

🔄 observe the software → record as expected value → test passes → bug stays

**If the expected value comes from what the software does now, the test agrees with the bug**

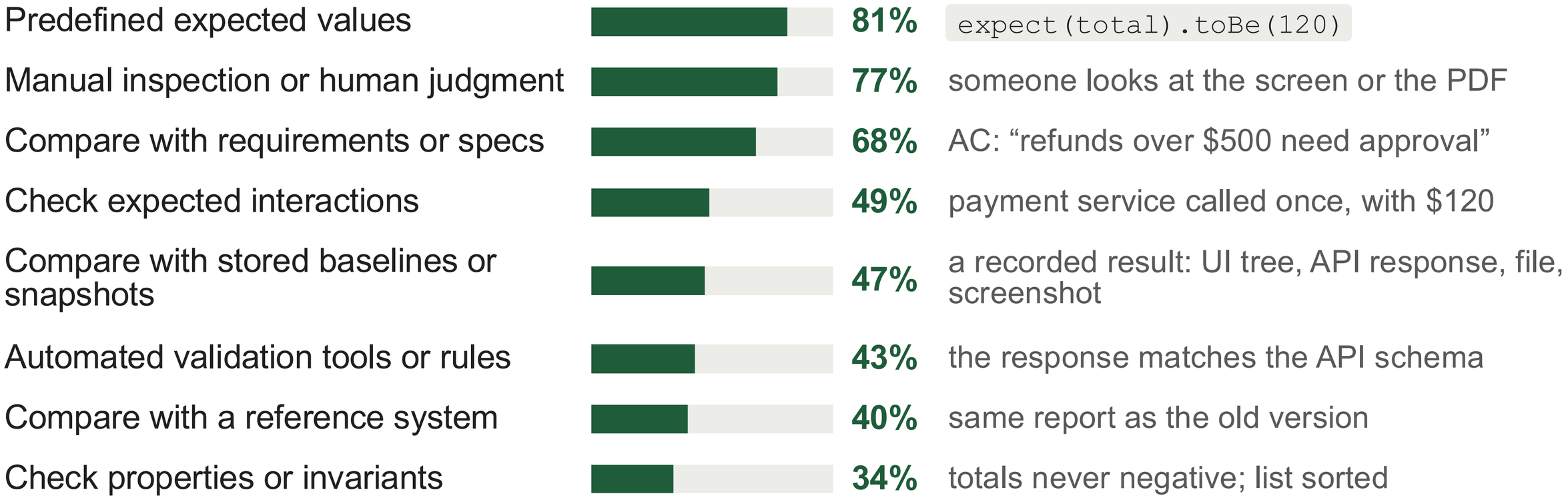
# E2E: What practitioners check




E2E tests · n = 47 · source: our study

**E2E oracles aren’t only visual: half also check backend state and logs.**

# E2E: How teams judge correctness




Rare: AI tools to validate outputs (17%)

E2E tests · n = 47 · source: our study

# Strong oracle ≠ strict oracle

Place an order for 2 items. Check the API response.

## ✗ Too strict

```
expect(res.body)
  .toEqual(
    savedResponse);
```

compares every field, even `id` and `createdAt`

✗ fails every run

## ✗ Too shallow

```
expect(res.status)
  .toBe(201);
```

wrong total or missing items still pass

✗ misses bugs

## ✓ Right

```
expect(res.body)
  .toMatchObject({
    status: "paid",
    total: 120,
    items: [{ sku: "A1" },
            { sku: "B2" }],
  });
```

checks only the fields the spec names

✓ checks the spec

**A good oracle checks the right things with the right precision . More checks, or stricter ones, don't make it better.**

# Same test, weak or strong check

| ✗ WEAK | ✓ STRONG | PROBLEM |
| --- | --- | --- |
| `expect(sendEmail).toHaveBeenCalled();` | `expect(sendEmail).toHaveBeenCalledWith("ana@x.com", expect.stringContaining("Order #101"));` | passes if the wrong person gets the email |
| `await expect(transfer(a, b, 500)).rejects.toThrow("insufficient funds");` | `await expect(transfer(a, b, 500)).rejects.toThrow("insufficient funds"); expect(balanceOf(a)).toBe(500);` | passes even if money moved before the error |
| `expect(items.length).toBe(3);` | `expect(items.map(i => i.sku)).toEqual(["A1", "B2", "C3"]);` | passes with the wrong items |
| `expect(fn).toThrow();` | `expect(fn).toThrow("max length exceeded");` | passes for any error, even a crash |

# GUI checks: weak or strong (1/2)

| ✗ WEAK | ✓ STRONG | PROBLEM |
| --- | --- | --- |
| `expect(toast).toBeVisible();` | `expect(toast)`<br>`  .toHaveText("Order saved");`<br>`await page.reload();`<br>`expect(row).toContainText("$120");` | nothing saved still passes |
| `await page`<br>`  .waitForTimeout(2000);`<br>`expect(await total`<br>`  .textContent())`<br>`  .toBe("$120");` | `// retries until it matches`<br>`await expect(total)`<br>`  .toHaveText("$120");` | slow and flaky |

# GUI checks: weak or strong (2/2)

| ✗ WEAK | ✓ STRONG | PROBLEM |
| --- | --- | --- |
| `page.locator(".btn-primary")` | `page.getByRole("button", { name: "Save" })` | breaks on any restyle |
| `await expect(page).toHaveScreenshot();` | `await expect(chart).toHaveScreenshot({ mask: [timestamp], maxDiffPixels: 100 });` | fails on every timestamp |

A person still reviews screenshot diffs before accepting a new baseline.





# Comparisons that mislead

## Snapshot updated without reading

**Snapshot** = a saved output used as the expected value; `-u` accepts every new output, bugs included

```
$ npx vitest -u
✓ 12 snapshots updated
```

**Fix:** review the snapshot diff like code

## Expected value copied from the code

```
// ran it once, pasted the result
expect(calcTax(order)).toBe(4.32);
```

**Fix:** compute it from the spec: 8% of $50 = $4.00

## An assertion that never runs

```
try { await save(bad); }
catch (e) { expect(e.code).toBe(400); }
```

**Fix:** `rejects.toThrow` or `expect.assertions(1)`

## Order that isn't guaranteed

```
expect(tags).toEqual(["a", "b"]);
```

**Fix:** sort first, or compare as a set

**A test can have an assertion and still check nothing.**

# Oracles drift and conflict

## Drift

the issue changes, the spec and the tests don't




**Issue #42, week 3:** free shipping over **$50** (was $75)

**Test, unchanged:**

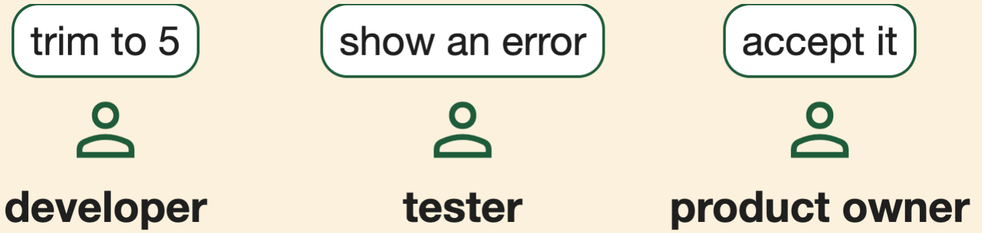
```
expect(shipping(order60)).toBe(5.99)
```

→ fails on correct code, or someone "fixes" the code back

## Conflict

developer, tester, and product owner each have their own "correct"

A user enters a 7-digit zip code:




→ the test encodes whoever wrote it

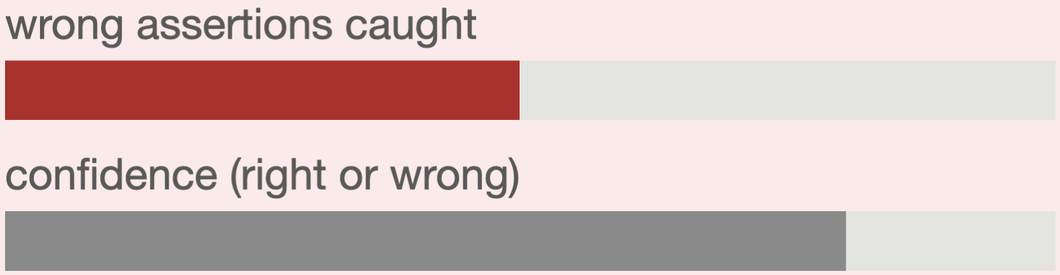
**What teams do:** ✓ settle it with the people who decide ✓ link each check to its source ✓ update tests when the requirement changes

# When AI writes the checks

**49%**

of wrong assertions caught

86 programmers caught **49%** of wrong AI-written assertions, with the **same confidence** whether right or wrong




> **“the same AI writing and running the tests is not trustworthy!”** — Outline team

## In review

- ✓ Try to **break** each assertion: find an input it gets wrong
- ✓ **Slow down** : wrong assertions get accepted fast
- ✓ Check it against the **spec** , not the AI’s explanation

Kaufman, Brun, Murali, Endres, “Programmers Are Poor and Overconfident Judges of LLM-Generated Assertions,” arXiv, 2026